



HCMUTE

TRƯỜNG ĐẠI HỌC

SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH

HCMC University of Technology and Education



KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN

NHẬP MÔN LẬP TRÌNH PYTHON (IPPA233277)

THƯ VIỆN PANDAS



MỤC TIÊU BÀI HỌC

1. Hiểu được thiệu thư viện pandas
2. Hiểu và vận dụng được các thao tác với Series
3. Hiểu và vận dụng được các thao tác với DataFrame



1. Giới thiệu thư viện pandas
2. Thao tác với Series
3. Thao tác với DataFrame



- Pandas là một thư viện Python cung cấp các cấu trúc dữ liệu nhanh, mạnh mẽ, linh hoạt và mang hàm ý
- Tên thư viện được bắt nguồn từ panel data (bảng dữ liệu). Pandas được thiết kế để làm việc dễ dàng và trực quan với dữ liệu có cấu trúc (dạng bảng, đa chiều, có tiềm năng không đồng nhất) và dữ liệu chuỗi thời gian
- Mục tiêu của pandas là trở thành khối căn bản (building block) cấp cao cơ bản cho công việc thực tế, phân tích dữ liệu thế giới thực trong Python, và rộng hơn là trở thành công cụ thao tác / phân tích mã nguồn mở mạnh mẽ và linh hoạt nhất có sẵn trong bất kỳ loại ngôn ngữ lập trình nào



- Cài đặt thư viện pandas
 - ✓ Sử dụng pip và gõ lệnh : `pip install pandas`
 - ✓ Hoặc bằng Anaconda, dùng lệnh : `conda install pandas`
- Khai báo thư viện pandas : `import pandas as pd`
- Pandas có hai cấu trúc dữ liệu cơ bản:
 - ✓ Series (1 chiều)
 - ✓ DataFrame (2 chiều)



THAO TÁC VỚI SERIES – TRUY CẬP

- Truy cập giá trị trong Series tương tự với ndarray trong numpy thông qua index và vị trí

Ví dụ:

```
data = {'a' : -1.3, 'b' : 11.7, 'd' : 2.0, 'f': 10, 'g': 5}
```

```
ser = pd.Series(data,index=['a','c','b','d','e','f'])
```

```
print(ser['d']) # 2.0
```

```
print(ser['c']) # nan
```

```
print(ser[:'d']) # lấy dữ liệu tới index = 'd' (1)
```

```
print(ser[:2]) # lấy 2 vị trí đầu (2)
```

```
print(ser[-3:]) # lấy 3 vị trí cuối (3)
```

```
a = np.asarray(ser) # chuyển đổi sang mảng
```

```
print(a) # [-1.3 nan 11.7 2. nan 10.]
```

```
a    -1.3  
c     NaN  
b    11.7  
d     2.0  
dtype: float64
```

(1)

```
a    -1.3  
c     NaN  
dtype: float64
```

(2)

```
d     2.0  
e     NaN  
f    10.0  
dtype: float64
```

(3)



THAO TÁC VỚI SERIES

- Series là một mảng một chiều giống như mảng numpy và có thêm một bảng đánh label.
- Series có thể được khởi tạo thông qua NumPy, kiểu Dict hoặc các dữ liệu vô hướng bình thường.
- Series có nhiều thuộc tính như index, array, values, dtype, v.v.
- Series có thể được tạo bằng cách lấy dữ liệu từ các kho dữ liệu (SQL Database, CSV, Excel, ...).

Khai báo

```
Series([data, index, dtype, name, copy, . . . ])
```

Ví dụ:

```
import pandas as pd  
s = pd.Series([0,1,2,3])  
print(s)
```

```
0    0  
1    1  
2    2  
3    3  
dtype: int64
```



THAO TÁC VỚI SERIES – KHỞI TẠO

- Tạo series với truyền index (1)

```
s = pd.Series([0,1,2,3], index=["a","b","c","d"])
```

- Tạo series từ dictionary (2)

```
data = {'a' : -1.3, 'b' : 11.7, 'd' : 2.0, 'f': 10, 'g': 5}
```

```
ser = pd.Series(data,index=['a','c','b','d','e','f'])
```

- Tạo series từ scalar (giá trị vô hướng), giá trị sẽ phù hợp với độ dài index (3)

```
ser = pd.Series(5, index=[1, 2, 3, 4, 5])
```

(1)

a	0
b	1
c	2
d	3

dtype: int64

(2)

a	-1.3
c	NaN
b	11.7
d	2.0
e	NaN
f	10.0

dtype: float64

(3)

1	5
2	5
3	5
4	5
5	5

dtype: int64



THAO TÁC VỚI DATAFRAME

- Là cấu trúc dữ liệu được gắn nhãn hai chiều với các cột và hàng như bảng tính (spreadsheet) hoặc bảng (table).
- Giống như Series, DataFrame có thể chứa bất kỳ loại dữ liệu nào.
- Trong DataFrame, tất cả các cột trong khung dữ liệu là series. Do đó, một DataFrame là sự kết hợp của nhiều Series đóng vai trò như các cột.

Khai báo:

```
DataFrame([data, index, columns, dtype, copy])
```



THAO TÁC VỚI DATAFRAME - KHỞI TẠO TỪ DỮ LIỆU

- Có thể khởi tạo dataframe thông qua việc đọc/xuất dữ liệu từ các tập tin (csv, txt, xls, xlsx, dat,...)
- Hàm hỗ trợ:
 - ✓ `pd.read_csv(file_name, index_col=0)` # đọc file vào dataframe
 - ✓ `df.info()` # cho biết thông tin định dạng và số lượng not-null của mỗi trường trong df
 - ✓ `df.dtypes` # kiểm tra định dạng dữ liệu của một trường
 - ✓ `df.describe()` # thống kê mô tả dữ liệu
 - ✓ `df.head/tail(n)` # số dòng hiển thị từ đầu/cuối
 - ✓ `df.to_csv/excel/json(file_name)` # lưu dataframe vào file



- **Tạo DataFrame từ Dictionary**

```
s = {'một': pd.Series([1., 2., 3., 5.], index=['a', 'b', 'c', 'e']),  
     'hai': pd.Series([1., 2., 3., 4.], index=['a', 'b', 'c', 'd'])}  
df = pd.DataFrame(s)  
df = pd.DataFrame(s, index=['a', 'c', 'd']) # lấy theo index được chọn
```

- **Tạo DataFrame từ List**

```
lst = ['Hello', 'Flinters', 'How', 'Are', 'You']  
df = pd.DataFrame(lst)
```

- **Tạo DataFrame từ Numpy Array**

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
df = pd.DataFrame(arr, columns = ['a', 'b', 'c'])
```



- Truy cập theo slice index: truyền vào index của dòng và cột, sử dụng hàm:

```
df.iloc[rows_slice, columns_slice] / df.loc[]
```

Ví dụ:

```
df.iloc[:5, :5]      # Lựa chọn 5 dòng đầu và 5 cột đầu của df
df.iloc[5:10, 2:4]    # Lựa chọn 5 dòng từ 5:10 và 2 cột từ 2:4
df.iloc[-5:, [1, 3]]  # Lựa chọn 5 dòng cuối và các cột 1 và 3
df.loc[10:15]         # Truy cập các dòng có index là 10:15
```



- Truy cập theo column names: sử dụng phổ biến bằng cách khai báo danh sách các cột cần truy xuất

```
df[[column_names]]
```

Ví dụ:

```
df[['col1', 'col2', 'col3']].head()      # lấy ra các trường từ bảng df
```

```
df[['col1', 'col2', 'col3']].iloc[10:15] # lấy ra các dòng từ 10:15 của các trường
```



- DataFrame có thể lọc dữ liệu thông qua các điều kiện đối với các trường.
- Điều kiện là một biểu thức logic trong dấu []
- Hàm hỗ trợ lọc dữ liệu:
 - ✓ `df[condition]` # đặt mỗi điều kiện trong (...) có thể dùng `&`, `|` (và, hoặc)
 - ✓ `df.filter()` # lọc dữ liệu
 - ✓ `df.sort_values()` # sắp xếp dataframe theo **by**
 - ✓ `df.select_dtypes()` # lọc theo kiểu dữ liệu



THAO TÁC VỚI DATAFRAME – HÀM

- Trên một trường dữ liệu của dataframe tích hợp các hàm tính toán hỗ trợ việc lọc và lấy dữ liệu
- Hàm hỗ trợ:
 - ✓ `df[column_name].min()/max()/mean()/median()/sum()`
 - ✓ `pd.merge(df1, df2, left_on, right_on, how)` # join dữ liệu 2 df
 - ✓ `pd.cut/qcut(x, bins, labels)` # phân chia giá trị của một trường vào những khoảng theo ngưỡng cắt
 - ✓ `df[columns_name].apply()` # biến đổi giá trị trường theo hàm số xác định trước
 - ✓ `df.map()` # biến đổi giá trị một biến sang giá trị mới dựa trên dictionary áp dụng
 - ✓ `df.melt([columns_name])` # giảm số trường hiển thị
 - ✓ `df.groupby()` / `pd.pivotable()` # thống kê theo nhóm
 - ✓ `df1.join(df2)` # tương đương `merge()`
 - ✓ `pd.concat()` # nối hai dữ liệu theo dòng hoặc cột
 - ✓ `df1.append(df2)` # nối dữ liệu theo dòng



- `df[column_name].plot(marker='o')` # biểu đồ dạng line
- `df.plot.bar()` # biểu đồ dạng cột
- `df.plot.pie()` # biểu đồ tròn
- `df.boxplot()` # biểu đồ boxplot
- `df.plot.area()` # biểu đồ vùng biểu diễn



THAO TÁC VỚI DATAFRAME – VÍ DỤ

Cho dữ liệu trong tập tin BostonHousing.csv.

✓ Đọc dữ liệu từ BostonHousing.csv

```
import pandas as pd
```

```
df = pd.read_csv("BostonHousing.csv", sep=";", header = 0, index_col = None)
```

```
df.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2



- ✓ Quan sát dữ liệu trong tập tin BostonHousing.csv: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   crim        506 non-null    float64
 1   zn           506 non-null    float64
 2   indus        506 non-null    float64
 3   chas         506 non-null    int64  
 4   nox          506 non-null    float64
 5   rm           506 non-null    float64
 6   age          506 non-null    float64
 7   dis          506 non-null    float64
 8   rad          506 non-null    int64  
 9   tax          506 non-null    int64  
10  ptratio      506 non-null    float64
11  b            506 non-null    float64
12  lstat        506 non-null    float64
13  medv         506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
None
```



THAO TÁC VỚI DATAFRAME – VÍ DỤ

✓ Quan sát dữ liệu trong tập tin BostonHousing.csv: `df.info()` hoặc `df.dtypes`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   crim        506 non-null   float64
 1   zn          506 non-null   float64
 2   indus       506 non-null   float64
 3   chas        506 non-null   int64  
 4   nox         506 non-null   float64
 5   rm          506 non-null   float64
 6   age         506 non-null   float64
 7   dis         506 non-null   float64
 8   rad         506 non-null   int64  
 9   tax         506 non-null   int64  
10  ptratio     506 non-null   float64
11  b           506 non-null   float64
12  lstat       506 non-null   float64
13  medv        506 non-null   float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
None
```

```
crim    float64
zn      float64
indus    float64
chas     int64
nox      float64
rm       float64
age      float64
dis      float64
rad      int64
tax      int64
ptratio  float64
b        float64
lstat    float64
medv     float64
dtype: object
```



THAO TÁC VỚI DATAFRAME – VÍ DỤ

✓ Thống kê mô tả dữ liệu: `df.describe()`

	crim	zn	indus	chas	nox	...	tax	ptratio	b	lstat	medv
count	506.000000	506.000000	506.000000	506.000000	506.000000	...	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	...	408.237154	18.455534	356.674032	12.653063	22.532806
std	8.601545	23.322453	6.860353	0.253994	0.115878	...	168.537116	2.164946	91.294864	7.141062	9.197104
min	0.006320	0.000000	0.460000	0.000000	0.385000	...	187.000000	12.600000	0.320000	1.730000	5.000000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	...	279.000000	17.400000	375.377500	6.950000	17.025000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	...	330.000000	19.050000	391.440000	11.360000	21.200000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	...	666.000000	20.200000	396.225000	16.955000	25.000000
max	88.976200	100.000000	27.740000	1.000000	0.871000	...	711.000000	22.000000	396.900000	37.970000	50.000000

[8 rows x 14 columns]

✓ Lựa chọn 5 dòng đầu và 5 cột đầu của dữ liệu: `df.iloc[:5, :5]`

	crim	zn	indus	chas	nox
0	0.00632	18.0	2.31	0	0.538
1	0.02731	0.0	7.07	0	0.469
2	0.02729	0.0	7.07	0	0.469
3	0.03237	0.0	2.18	0	0.458
4	0.06905	0.0	2.18	0	0.458



✓ Thống kê mô tả dữ liệu: `df.describe()`

	crim	zn	indus	chas	nox	...	tax	ptratio	b	lstat	medv
count	506.000000	506.000000	506.000000	506.000000	506.000000	...	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	...	408.237154	18.455534	356.674032	12.653063	22.532806
std	8.601545	23.322453	6.860353	0.253994	0.115878	...	168.537116	2.164946	91.294864	7.141062	9.197104
min	0.006320	0.000000	0.460000	0.000000	0.385000	...	187.000000	12.600000	0.320000	1.730000	5.000000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	...	279.000000	17.400000	375.377500	6.950000	17.025000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	...	330.000000	19.050000	391.440000	11.360000	21.200000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	...	666.000000	20.200000	396.225000	16.955000	25.000000
max	88.976200	100.000000	27.740000	1.000000	0.871000	...	711.000000	22.000000	396.900000	37.970000	50.000000

[8 rows x 14 columns]



THAO TÁC VỚI DATAFRAME – VÍ DỤ

- ✓ Lựa chọn 5 dòng đầu và 5 cột đầu của dữ liệu: `df.iloc[:5, :5]`
- ✓ Lựa chọn 5 dòng từ 5:10 và 2 cột từ 2:4: `df.iloc[5:10, 2:4]`
- ✓ Lấy ra các dòng từ 10:15 của các trường 'crim', 'tax', 'rad': `df[['crim', 'tax', 'rad']].iloc[10:15]`

	crim	zn	indus	chas	nox
0	0.00632	18.0	2.31	0	0.538
1	0.02731	0.0	7.07	0	0.469
2	0.02729	0.0	7.07	0	0.469
3	0.03237	0.0	2.18	0	0.458
4	0.06905	0.0	2.18	0	0.458

	indus	chas
5	2.18	0
6	7.87	0
7	7.87	0
8	7.87	0
9	7.87	0

	crim	tax	rad
10	0.22489	311	5
11	0.11747	311	5
12	0.09378	311	5
13	0.62976	307	4
14	0.63796	307	4



THAO TÁC VỚI DATAFRAME – VÍ DỤ

✓ Lọc ra các thị trấn mà có số phòng ở trung bình trên căn hộ là trên 4 và thuế suất trên 250

```
df[(df['rm']>4) & (df['tax']>250)].head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
6	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.60	12.43	22.9
7	0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311	15.2	396.90	19.15	27.1
8	0.21124	12.5	7.87	0	0.524	5.631	100.0	6.0821	5	311	15.2	386.63	29.93	16.5
9	0.17004	12.5	7.87	0	0.524	6.004	85.9	6.5921	5	311	15.2	386.71	17.10	18.9

✓ Lọc ra các trường có kiểu dữ liệu float: `df.select_dtypes('float').head()`

	crim	zn	indus	nox	rm	age	dis	ptratio	b	lstat	medv
0	0.00632	18.0	2.31	0.538	6.575	65.2	4.0900	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.469	6.421	78.9	4.9671	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.469	7.185	61.1	4.9671	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.458	6.998	45.8	6.0622	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.458	7.147	54.2	6.0622	18.7	396.90	5.33	36.2



THAO TÁC VỚI DATAFRAME – VÍ DỤ

✓ Sắp xếp dữ liệu giá trị của căn nhà theo chiều giảm dần:

```
df.sort_values('medv', ascending = False).head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
369	5.66998	0.0	18.1	1	0.631	6.683	96.8	1.3567	24	666	20.2	375.33	3.73	50.0
370	6.53876	0.0	18.1	1	0.631	7.016	97.5	1.2024	24	666	20.2	392.05	2.96	50.0
371	9.23230	0.0	18.1	0	0.631	6.216	100.0	1.1691	24	666	20.2	366.15	9.53	50.0
372	8.26725	0.0	18.1	1	0.668	5.875	89.6	1.1296	24	666	20.2	347.88	8.88	50.0
368	4.89822	0.0	18.1	0	0.631	4.970	100.0	1.3325	24	666	20.2	375.52	3.26	50.0

✓ Sắp xếp đồng thời giá trị của căn nhà và giá trị thuế suất :

```
df.sort_values(['medv', 'tax'], ascending = False).head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv
368	4.89822	0.0	18.1	0	0.631	4.970	100.0	1.3325	24	666	20.2	375.52	3.26	50.0
369	5.66998	0.0	18.1	1	0.631	6.683	96.8	1.3567	24	666	20.2	375.33	3.73	50.0
370	6.53876	0.0	18.1	1	0.631	7.016	97.5	1.2024	24	666	20.2	392.05	2.96	50.0
371	9.23230	0.0	18.1	0	0.631	6.216	100.0	1.1691	24	666	20.2	366.15	9.53	50.0
372	8.26725	0.0	18.1	1	0.668	5.875	89.6	1.1296	24	666	20.2	347.88	8.88	50.0



✓ Tính toán như min, max, mean, median, sum để tính các giá trị đặc trưng cho từng trường:

```
df['tax'].min(), df['tax'].max(), df['tax'].mean(), df['tax'].median(), df['tax'].sum()
```

```
# 187 711 408.2371541501976 330.0 206568
```

✓ Phân chia giá trị của một trường liên tục vào những khoảng theo ngưỡng cắt:

```
bins = [-999999, 250, 400, 999999]
```

```
labels = ['low', 'normal', 'high']
```

```
# low: -999999 <- 250
```

```
# normal: 250 <- 400
```

```
# high: 400 <- 999999
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	medv	tax_labels
54	0.01360	75.0	4.00	0	0.410	5.888	47.6	7.3197	3	469	21.1	396.90	14.80	18.9	high
111	0.10084	0.0	10.01	0	0.547	6.715	81.6	2.6775	6	432	17.8	395.59	10.16	22.8	high
112	0.12329	0.0	10.01	0	0.547	5.913	92.9	2.3534	6	432	17.8	394.95	16.21	18.8	high
113	0.22212	0.0	10.01	0	0.547	6.092	95.4	2.5480	6	432	17.8	396.90	17.09	18.7	high
114	0.14231	0.0	10.01	0	0.547	6.254	84.2	2.2565	6	432	17.8	388.74	10.45	18.5	high

```
df['tax_labels'] = pd.cut(df['tax'], bins=bins, labels=labels)
```

```
print(df[df['tax_labels']=='high'].head())
```



THAO TÁC VỚI DATAFRAME – VÍ DỤ

- ✓ Sử dụng hàm `qcut` khi không muốn chia các bin dựa vào ngưỡng mà chỉ muốn khai báo số lượng bins và để cho hàm số tự quyết định ngưỡng để chia đều các quan sát vào các bins:

```
import numpy as np

labels = ['low', 'normal', 'high']

tax_labels = pd.qcut(df['tax'], q=3, labels=labels)

print(np.unique(tax_labels, return_counts = True))

# (array(['high', 'low', 'normal'], dtype=object), array([168, 172, 166]))
```

- ✓ Nhân đôi giá trị của `tax` thì có thể sử dụng hàm `apply` với `lambda`:

```
df['tax'].apply(lambda x: 2*x).head()
```

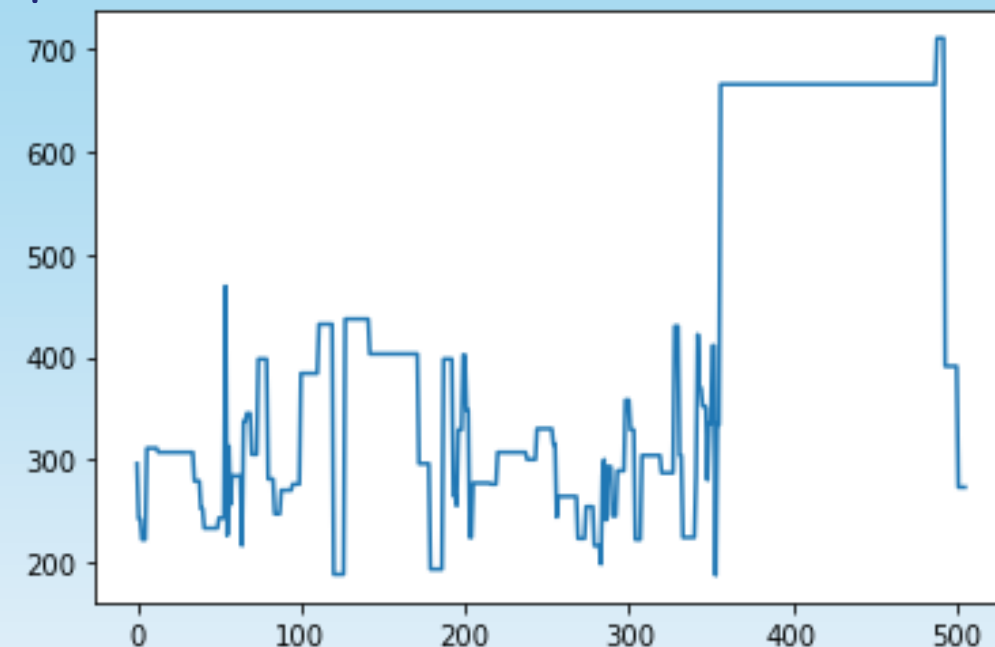


THAO TÁC VỚI DATAFRAME – VÍ DỤ

✓ Map các giá trị của trường `df['tax_labels']` sang các giá trị tiếng Việt:

```
dict_tax = {  
    'low': 'thap',  
    'normal': 'tb',  
    'high': 'cao'  
}  
  
print(df['tax_labels'].map(dict_tax).head())
```

✓ Vẽ biểu đồ đường cho trường thuế (tax): `df['tax'].plot()`

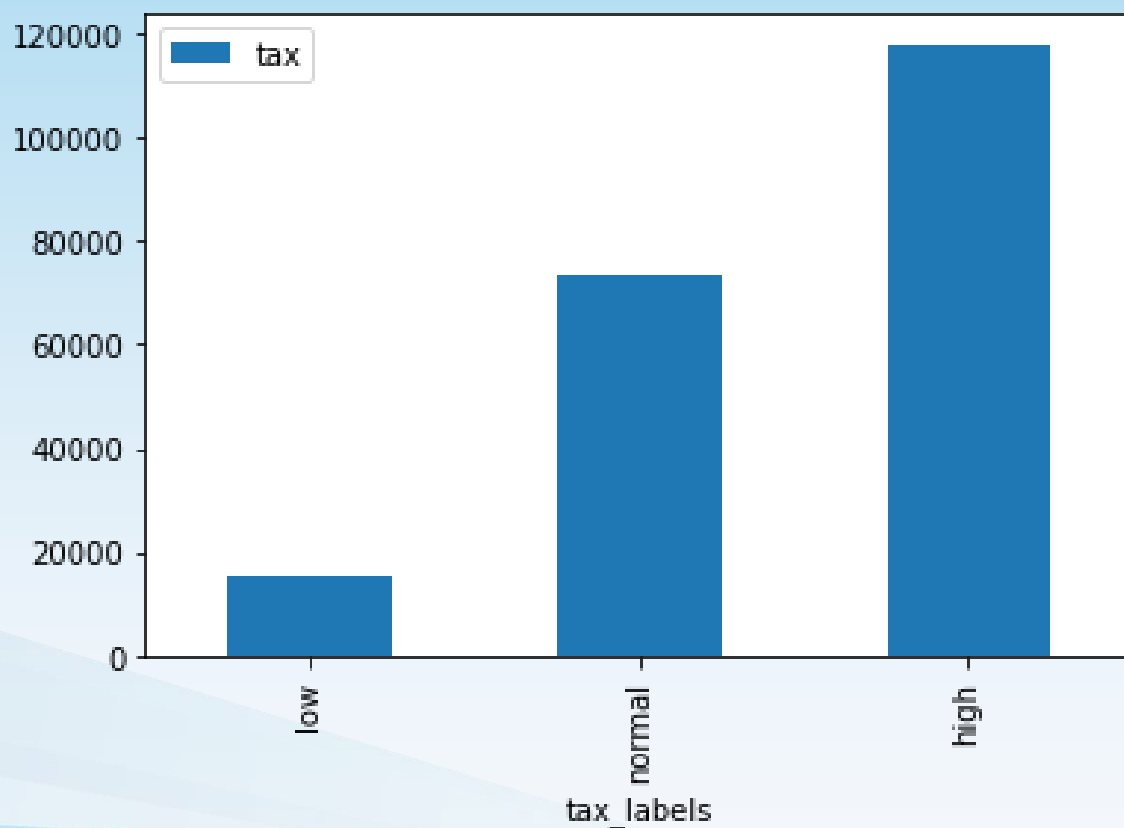


THAO TÁC VỚI DATAFRAME – VÍ DỤ

✓ Vẽ biểu đồ cột so sánh chênh lệch giữa các nhóm về mặt giá trị tuyệt đối:

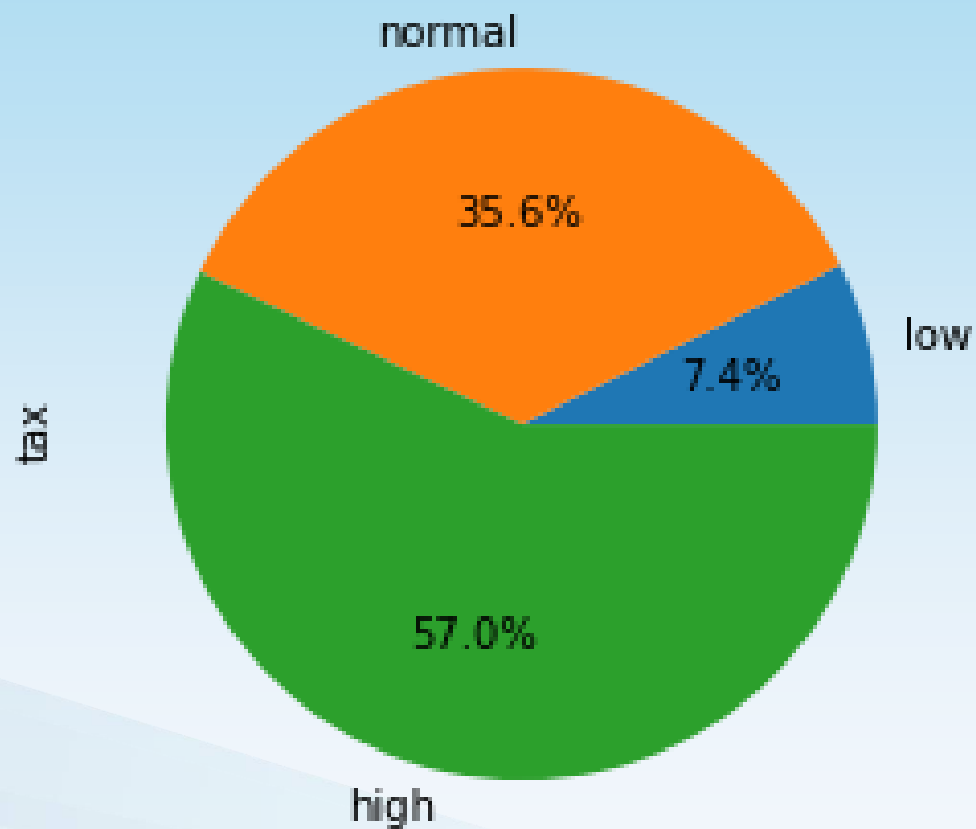
```
df_summary = df[['tax_labels', 'tax']].groupby('tax_labels').sum()
```

```
df_summary.plot.bar()
```



✓ Vẽ biểu đồ tròn dùng để thể hiện giá trị phần trăm. Phù hợp khi so sánh giá trị tương đối giữa các nhóm.:

```
df_summary['tax'].plot.pie(autopct = '%1.1f%%')
```



1. Điền ... để tạo Series trong Pandas?

`pd....(mylist)`

`[Series]`

2. Điền ... để trả về giá trị đầu tiên trong chuỗi Pandas có tên myseries?

`myseries...`

`[[0]]`

3. Điền ... để thêm nhãn x, y, z cho Pandas Series?

`pd.Series(mylist, ... )`

`[index = ["x", "y", "z"]]`

4. Điền ... để tạo DataFrame trong Pandas?

`pd....(data)`

`[DataFrame]`

5. Điền ... để trả về dòng đầu tiên trong DataFrame?

`df....`

`[loc[0]]`

2. Điền ... nạp dữ liệu từ tập tin CSV vào trong DataFrame

`df....(data)`

`[read_csv]`

3. Cú pháp để trả về toàn bộ dữ liệu có trong DataFrame

`df....`

`[to_string()]`

4. Điền ... để nạp dữ liệu JSON vào trong DataFrame?

`df....(data)`

`[read_json]`



```
df['Duration'].plot()
```

5. Điền ... nạp dữ liệu dictionary data của Python vào trong DataFrame?

```
pd....(data)
```

```
[DataFrame]
```

6. Cú pháp để thay thế ô rỗng bằng giá trị “130”?

```
df....(130)
```

```
[fillna]
```

7. Giá trị ± 0.9 được xem là tương quan tốt?

A. Đúng

B. Sai

8. Cú pháp để trực quan hóa dữ liệu trong DataFrame như một biểu đồ?

```
df....()
```

```
[plot]
```

Nhập môn lập trình Python (IPPA23327)

9. Điền ... để xác định biểu đồ phải thuộc loại “scatter”?

```
df.plot(..., x = 'Duration', y = 'Calories')
```

```
[kind = 'scatter']
```

10. Điền ... để xác định biểu đồ phải thuộc loại “scatter”?

```
df['Duration'].plot(...)
```

```
[kind = 'hist']
```

11.

